1. What is Python? Explain its features.

Answer:
Python is a high-level, interpreted, general-purpose programming language known for its simplicity and readability. It supports multiple programming paradigms like procedural, object-oriented, and functional programming.

Features:

- **Easy to learn and use:** Python's clear syntax makes it accessible.
- **Interpreted language:** Code executes line-by-line, easing debugging.
- **Dynamically typed:** Variable types are determined at runtime.
- **Extensive standard library:** Offers built-in modules for diverse tasks.
- **Portability:** Runs on multiple platforms without modification.
- **Open-source:** Free to use and extend.

---

2. What are Python's data types?

Answer:
Python provides several built-in data types to store different kinds of data:

- **Numeric types:** int, float, complex
- **Sequence types:** list, tuple, range
- **Text type:** str
- **Mapping type:** dict
- **Set types:** set, frozenset
- **Boolean type:** bool
- **None type:** NoneType representing the absence of a value.

---

3. What are lists, tuples, sets, and dictionaries? How do they differ?

Answer:

- **List:** Ordered, mutable collection of items.

- my_list = [1, 2, 3]
- my_list[0] = 10 # Allowed
- **Tuple:** Ordered, immutable collection of items.
- my_tuple = (1, 2, 3)
- # my_tuple[0] = 10 # Not allowed (immutable)
- **Set:** Unordered, mutable collection of unique elements.
- my_set = {1, 2, 3}
- my_set.add(4)
- **Dictionary:** Unordered, mutable collection of key-value pairs.
- my_dict = {'a': 1, 'b': 2}
- my_dict['c'] = 3

Differences:

- Lists and tuples are ordered; sets and dictionaries are unordered.
- Lists and sets are mutable; tuples are immutable.
- Dictionaries store key-value pairs; others store single values.

---

4. What is the difference between mutable and immutable objects?

Answer:

- **Mutable objects** can be changed after creation (e.g., lists, sets, dictionaries).
- **Immutable objects** cannot be changed once created (e.g., integers, floats, strings, tuples).

Example:

my_list = [1, 2, 3]

my_list[0] = 10 # Works

my_tuple = (1, 2, 3)

```
# my_tuple[0] = 10  # Raises error
```

---

5. What are Python variables? How is memory managed in Python?

Answer:
Variables in Python are references (labels) to objects stored in memory. Python uses **automatic memory management** with reference counting and **garbage collection** to reclaim unused objects.

Example:

```
x = 10

y = x
```

Here, x and y both reference the same integer object 10.

---

6. What are Python functions? Explain *args and **kwargs.

Answer:
A **function** is a reusable block of code that performs a task.

- *args allows a function to accept any number of positional arguments as a tuple.
- **kwargs allows a function to accept any number of keyword arguments as a dictionary.

Example:

```
def example_func(*args, **kwargs):
    print("Positional arguments:", args)
    print("Keyword arguments:", kwargs)


example_func(1, 2, a=3, b=4)
```

---

7. What are lambda functions? Provide an example.

Answer:
A **lambda function** is an anonymous, inline function defined with the lambda keyword. It can have any number of arguments but only one expression.

Syntax:

lambda arguments: expression

Example:

```
add = lambda x, y: x + y

print(add(3, 4))  # Output: 7
```

---

8. What are Python decorators? How do they work?

Answer:
A **decorator** is a function that modifies or enhances another function without changing its code.

Example:

```
def decorator_func(func):
    def wrapper():
        print("Before function call")
        func()
        print("After function call")
    return wrapper


@decorator_func
def say_hello():
    print("Hello!")


say_hello()
```

Output:

```
Before function call
Hello!
After function call
```

---

9. What is a generator? How is it different from a list?

Answer:
A **generator** is an iterable that generates values on the fly using the yield keyword, saving memory. Unlike lists, generators do not store all values in memory.

Example:

```
def gen():
    yield 1
    yield 2
    yield 3

for val in gen():
    print(val)
```

---

10. What are list comprehensions? Provide an example.

Answer:
List comprehensions provide a concise way to create lists using a single line of code.

Syntax:

```
[expression for item in iterable if condition]
```

Example:

```
squares = [x**2 for x in range(5)]
print(squares)  # Output: [0, 1, 4, 9, 16]
```

11. Explain Python's pass, continue, and break statements.

Answer:

- **pass:** Does nothing; used as a placeholder for future code.
- **continue:** Skips the current iteration and moves to the next loop iteration.
- **break:** Terminates the loop immediately.

Example:

```
for i in range(5):
    if i == 2:
        continue  # Skip when i is 2
    if i == 4:
        break  # Exit loop when i is 4
    print(i)
```

Output:

```
0
1
3
```

---

12. How is error handling done in Python? Explain try, except, and finally.

Answer:
Python uses try-except blocks to handle exceptions and prevent crashes. The finally block executes code regardless of whether an exception occurred.

Syntax:

```
try:
    # Code that may raise an exception
except ExceptionType:
    # Handle exception
finally:
    # Always execute this
```

Example:

```
try:
    x = 1 / 0
except ZeroDivisionError:
```

```
    print("Cannot divide by zero.")
finally:
    print("Execution completed.")
```

---

13. What are Python modules and packages?

Answer:

- **Module:** A single Python file containing definitions and statements.
- **Package:** A collection of modules organized in directories containing a special __init__.py file.

---

14. How do you import modules in Python?

Answer:
Modules are imported using the import statement.

Examples:

```
import math
from math import sqrt
import math as m
```

---

15. What is the difference between shallow copy and deep copy?

Answer:

- **Shallow copy:** Copies the reference of nested objects; changes to nested objects affect both copies.
- **Deep copy:** Copies nested objects recursively, creating independent copies.

Example:

```
import copy
list1 = [[1, 2], [3, 4]]
shallow = copy.copy(list1)
```

```
deep = copy.deepcopy(list1)
```

---

16. What is the difference between == and is operators?

Answer:

- == checks for **value equality** (whether objects have the same content).
- is checks for **identity equality** (whether objects are the same in memory).

---

17. What are Python classes and objects? Explain OOP concepts in Python.

Answer:

- **Class:** Blueprint for creating objects.
- **Object:** Instance of a class.

OOP Concepts:

- **Encapsulation:** Bundling data and methods.
- **Inheritance:** Deriving new classes from existing ones.
- **Polymorphism:** Using a unified interface for different data types.
- **Abstraction:** Hiding complex implementation details.

---

18. What is inheritance? Explain with an example.

Answer:
Inheritance allows a class (child) to inherit attributes and methods from another class (parent).

Example:

```
class Animal:
    def sound(self):
        print("Animal sound")


class Dog(Animal):
```

```
    def sound(self):
        print("Bark")


dog = Dog()
dog.sound()  # Output: Bark
```

---

19. What are Python's built-in functions? Name some important ones.

Answer:
Python has many built-in functions like print(), len(), type(), int(), str(), input(), range(), sum(), max(), min(), sorted(), etc.

---

20. How does Python manage memory? What is garbage collection?

Answer:
Python manages memory automatically through reference counting and a **garbage collector** that frees memory occupied by objects no longer in use.

21. What is the Global Interpreter Lock (GIL) in Python?

Answer:
The **Global Interpreter Lock (GIL)** is a mutex that protects access to Python objects, preventing multiple native threads from executing Python bytecodes simultaneously. This means Python threads cannot run in true parallelism in a single process, which affects CPU-bound multithreading performance.

---

22. What are iterators and iterables in Python?

Answer:

- **Iterable:** An object capable of returning its members one at a time (e.g., list, tuple, string).
- **Iterator:** An object representing a stream of data returned by the iter() function; it implements the __next__() method.

Example:

```
my_list = [1, 2, 3]

iterator = iter(my_list)

print(next(iterator))  # Output: 1
```

---

23. What is the difference between range and xrange? (Python 2)

Answer:

- **range:** Returns a list.
- **xrange:** Returns an iterator, more memory efficient.
  Note: In Python 3, range() behaves like xrange() in Python 2.

---

24. How do you handle files in Python? Explain reading and writing files.

Answer:
Use the built-in open() function with different modes: 'r' for reading, 'w' for writing, 'a' for appending.

Example:

```
with open('file.txt', 'r') as f:

    data = f.read()


with open('file.txt', 'w') as f:

    f.write('Hello, World!')
```

---

25. What are Python's comprehension techniques (list, dict, set)?

Answer:
Comprehensions provide concise ways to create collections.

- **List comprehension:**
- squares = [x**2 for x in range(5)]
- **Dict comprehension:**

- square_dict = {x: x**2 for x in range(5)}
- **Set comprehension:**
- square_set = {x**2 for x in range(5)}

---

26. Explain the difference between staticmethod, classmethod, and instance methods.

Answer:

- **Instance method:** Operates on an instance, takes self as the first parameter.
- **Class method:** Bound to class, takes cls as the first parameter, defined with @classmethod.
- **Static method:** Doesn't access class or instance, behaves like a regular function inside a class, defined with @staticmethod.

---

27. What is multithreading and multiprocessing in Python?

Answer:

- **Multithreading:** Running multiple threads in a single process; limited by GIL for CPU-bound tasks but useful for I/O-bound tasks.
- **Multiprocessing:** Running multiple processes; bypasses GIL and enables true parallelism for CPU-bound tasks.

---

28. What are Python's data structures?

Answer:
Common data structures include:

- Lists
- Tuples
- Sets
- Dictionaries
- Queues (from collections module)

- Stacks (can be implemented using lists)

---

29. What is duck typing in Python?

Answer:
Duck typing means an object's suitability is determined by the presence of methods and properties, rather than the actual type.

Example:
"If it looks like a duck and quacks like a duck, it's a duck."

---

30. How do you debug Python code?

Answer:
You can use tools like:

- print() statements
- Built-in pdb module (Python Debugger)
- IDE debuggers (PyCharm, VS Code)
- Third-party tools like ipdb and pudb.

31. What are Python's standard libraries you have used?

Answer:
Python has a rich standard library including modules for various tasks such as:

- **os** (interacting with the operating system)
- **sys** (system-specific parameters)
- **math** (mathematical functions)
- **datetime** (date and time manipulation)
- **json** (JSON parsing and serialization)
- **re** (regular expressions)
- **collections** (specialized data structures)
- **random** (random number generation)

- **subprocess** (running system commands)
- **threading/multiprocessing** (concurrency)

---

32. What is a Python virtual environment and why is it used?

Answer:
A **virtual environment** is an isolated environment for Python projects that allows separate package installations for different projects, avoiding dependency conflicts.

Usage:

```
python -m venv env

source env/bin/activate  # Linux/Mac

env\Scripts\activate  # Windows
```

---

33. How do you handle exceptions? What are custom exceptions?

Answer:
Exceptions are handled using try-except blocks. You can define **custom exceptions** by subclassing the Exception class.

Example:

```
class MyError(Exception):
    pass


try:
    raise MyError("Something went wrong")
except MyError as e:
    print(e)
```

---

34. What is Python slicing?

Answer:
Slicing extracts parts of sequences using [start:stop:step].

Example:

```
lst = [0,1,2,3,4,5]

print(lst[1:4])  # Output: [1,2,3]

print(lst[::-1])  # Output: [5,4,3,2,1,0]
```

---

35. What is the difference between list and array in Python?

Answer:

- **List:** Built-in, can hold heterogeneous elements.
- **Array:** From the array module; stores elements of the same type for memory efficiency.

---

36. What is the difference between local, global, and nonlocal variables?

Answer:

- **Local:** Defined inside a function, accessible only there.
- **Global:** Defined at the module level, accessible anywhere in the module.
- **Nonlocal:** Refers to variables in the nearest enclosing function scope (used in nested functions).

---

37. What are Python's control flow statements?

Answer:
Python uses if, elif, else, loops (for, while), and control statements like break, continue, and pass.

---

38. What is the purpose of the yield keyword?

Answer:
yield is used inside a function to make it a generator. It returns a value and pauses the function, resuming later to produce a sequence lazily.

---

39. How do you optimize Python code?

Answer:

- Use built-in functions and libraries
- Avoid unnecessary computations
- Use list comprehensions and generators
- Profile code with cProfile
- Use concurrency for I/O-bound tasks
- Use efficient data structures

---

40. What are Python's magic methods?

Answer:
Magic methods are special methods with double underscores like __init__, __str__, __repr__, __add__, that let you define object behavior for built-in operations.

41. Explain Python's object lifecycle and garbage collection.

Answer:
The lifecycle of a Python object includes creation, usage, and destruction. Python uses **reference counting** to track how many references point to an object. When the reference count reaches zero, the object is destroyed. Additionally, Python's **garbage collector** handles cyclic references that reference counting alone can't resolve.

---

42. What is the difference between bytes, str, and unicode in Python?

Answer:

- **str:** Text data type in Python 3 (Unicode).
- **bytes:** Sequence of byte (8-bit) values, used for binary data.
- **unicode:** In Python 2, unicode represented text strings; in Python 3, all strings are Unicode by default.

---

43. How do you manage dependencies in Python projects?

Answer:
Dependencies are managed using:

- requirements.txt files with pip install -r requirements.txt
- Virtual environments to isolate dependencies
- Tools like pipenv or poetry for advanced dependency management.

---

44. What is the difference between map(), filter(), and reduce()?

Answer:

- **map():** Applies a function to all items in an iterable.
- **filter():** Filters items based on a function returning True/False.
- **reduce():** Applies a rolling computation to sequential pairs in an iterable (from functools module).

---

45. How do you work with JSON in Python?

Answer:
Using the json module:

- Convert JSON string to Python object: json.loads()
- Convert Python object to JSON string: json.dumps()
- Read/write JSON files using json.load() and json.dump().

---

46. What is the Python iterator protocol?

Answer:
An object is an iterator if it implements two methods: __iter__() returning the iterator object itself, and __next__() returning the next item or raising StopIteration when done.

---

47. How do you handle concurrency in Python?

Answer:
Python supports concurrency using:

- **Multithreading:** For I/O-bound tasks (subject to GIL).
- **Multiprocessing:** For CPU-bound tasks to achieve parallelism.
- **Asyncio:** Asynchronous programming for handling large numbers of connections.

---

48. What are Python's metaclasses?

Answer:
Metaclasses are classes of classes that define how classes behave. They allow customization of class creation and are specified using the metaclass keyword in class definitions.

---

49. What is a closure in Python?

Answer:
A closure is a nested function that remembers and has access to variables in the enclosing scope even after the outer function has finished execution.

---

50. What is the difference between Python 2 and Python 3?

Answer:
Key differences:

- Python 3 uses print() as a function.
- Strings are Unicode by default in Python 3.
- Python 3 has improved integer division behavior.
- Python 2 is deprecated; Python 3 is the current standard.

51. How do you test Python code? Explain unittest or pytest frameworks.

Answer:

- **unittest:** Built-in testing framework based on test cases and test suites.
- **pytest:** Third-party, more powerful and simpler, supports fixtures and parametrized tests.

Example with unittest:

```
import unittest


class TestMath(unittest.TestCase):
    def test_add(self):
        self.assertEqual(2 + 3, 5)


if __name__ == '__main__':
    unittest.main()
```

---

52. How is memory allocated for Python objects?

Answer:
Python objects are stored in **heap memory**. Each object has a reference count, and memory is managed via reference counting and garbage collection. Small integers and interned strings are cached for efficiency.

---

53. What is memoization? How do you implement it in Python?

Answer:
Memoization is an optimization technique to cache function results to avoid repeated computations.

Example:

```
def fib(n, memo={}):
    if n in memo:
        return memo[n]
    if n <= 1:
        return n
    memo[n] = fib(n-1, memo) + fib(n-2, memo)
```

```
    return memo[n]
```

---

54. Explain Python's data encapsulation and access specifiers.

Answer:
Python uses naming conventions for encapsulation:

- **Public:** Accessible everywhere (var)
- **Protected:** Indicated by a single underscore _var (convention only)
- **Private:** Indicated by double underscore __var (name mangling to restrict access)

---

55. What is the difference between *args and **kwargs?

Answer:

- *args collects extra positional arguments as a tuple.
- **kwargs collects extra keyword arguments as a dictionary.

---

56. How do you create a singleton class in Python?

Answer:
By controlling instance creation to ensure only one instance exists.

Example using a decorator:

```
def singleton(cls):
    instances = {}
    def get_instance(*args, **kwargs):
        if cls not in instances:
            instances[cls] = cls(*args, **kwargs)
        return instances[cls]
    return get_instance


@singleton
```

```
class MyClass:
    pass
```

---

57. What is the difference between mutable and immutable types?

Answer:

- **Mutable:** Can be changed after creation (list, dict, set).
- **Immutable:** Cannot be changed (int, float, tuple, str).

---

58. Explain Python's async and await keywords.

Answer:
Used for asynchronous programming to write non-blocking code. async defines a coroutine; await pauses coroutine execution until awaited task completes.

---

59. What are coroutines in Python?

Answer:
Coroutines are generalizations of generators used for cooperative multitasking and asynchronous programming.

---

60. How do you serialize and deserialize objects in Python?

Answer:
Use the pickle module.

Example:

```
import pickle


data = {'a': 1}

serialized = pickle.dumps(data)

deserialized = pickle.loads(serialized)
```

61. What is the difference between a module and a package?

Answer:

- **Module:** A single .py file containing Python code (functions, classes, variables).
- **Package:** A directory containing multiple modules and an __init__.py file, used to organize modules hierarchically.

---

62. How does Python's import system work?

Answer:
Python searches for modules in the following order:

1. Current directory
2. Directories in the PYTHONPATH environment variable
3. Standard library directories
4. Installed third-party packages

Using import loads the module and executes its code once per session.

---

63. What is monkey patching?

Answer:
Monkey patching is dynamically modifying or extending a module or class at runtime without changing the original source code.

Example:

```
import math


def new_sqrt(x):
    return "patched!"


math.sqrt = new_sqrt
```

```
print(math.sqrt(4))  # Output: patched!
```

---

64. What is the difference between Python's list and NumPy array?

Answer:

- **List:** General-purpose, can store heterogeneous data, slower for numeric operations.
- **NumPy array:** Homogeneous, optimized for numerical computations, supports vectorized operations and uses less memory.

---

65. How do you handle memory leaks in Python?

Answer:
Memory leaks can occur due to lingering references or circular references. Use tools like gc module to detect and break cycles, and ensure objects are properly dereferenced.

---

66. What is the purpose of the with statement?

Answer:
with ensures proper acquisition and release of resources, like files or locks, using context managers.

Example:

```
with open('file.txt', 'r') as f:
    data = f.read()
```

Automatically closes the file after the block.

---

67. How do you manage Python dependencies in production?

Answer:
Using virtual environments, requirements.txt, and dependency managers like pipenv or poetry to isolate and track dependencies.

---

68. Explain the difference between Python's sort() and sorted().

Answer:

- sort() modifies the list in place and returns None.
- sorted() returns a new sorted list, leaving the original unchanged.

---

69. What are Python's descriptors?

Answer:
Descriptors are objects that customize attribute access using methods like __get__, __set__, and __delete__.

---

70. What is metaprogramming?

Answer:
Metaprogramming means writing code that manipulates code, like creating classes or functions dynamically, often using metaclasses or decorators.

71. Explain Python's context managers.

Answer:
Context managers handle setup and cleanup actions automatically using the with statement. They implement the __enter__() and __exit__() methods.

Example:

```
class MyContext:
    def __enter__(self):
        print("Enter")
        return self
    def __exit__(self, exc_type, exc_value, traceback):
        print("Exit")

with MyContext():
    print("Inside")
```

---

72. How is exception propagation handled in Python?

Answer:
If an exception is not caught in the current block, it propagates up to the calling functions until it is handled or terminates the program.

---

73. What is the difference between __str__ and __repr__ methods?

Answer:

- __str__: Provides a readable, user-friendly string representation (used by print()).
- __repr__: Provides an official string representation that can be used to recreate the object (used in debugging).

---

74. What are Python's built-in functions related to iterators?

Answer:
Key functions include iter(), next(), enumerate(), zip(), and reversed().

---

75. How do you manage environment variables in Python?

Answer:
Using the os module:

```
import os

value = os.getenv('MY_VAR', 'default_value')
```

---

76. What is the purpose of the global keyword?

Answer:
It allows a function to modify a variable defined in the global scope.

---

77. What is the difference between class variables and instance variables?

Answer:

- **Class variables:** Shared across all instances.

- **Instance variables:** Unique to each instance.

---

78. How do you implement method overloading in Python?

Answer:
Python does not support method overloading by default. It can be simulated by checking argument types inside a method or using default arguments.

---

79. How do you implement method overriding?

Answer:
Subclass can provide its own implementation of a method from the parent class.

---

80. Explain Python's data hiding.

Answer:
Using name mangling with double underscores __var to make variables less accessible from outside the class.

81. What is the difference between is and == operators?

Answer:

- is checks **object identity** (whether two references point to the same object in memory).
- == checks **value equality** (whether two objects have the same value).

---

82. How do you perform unit testing in Python?

Answer:
Using the unittest framework, you write test cases by subclassing unittest.TestCase and defining methods that test functionality.

---

83. What are Python's built-in types?

Answer:
Common built-in types include: int, float, complex, bool, str, list, tuple, set, dict, range, bytes.

---

84. What is the difference between mutable and immutable data types?

Answer:

- **Mutable:** Objects that can be changed after creation (e.g., lists, dicts).
- **Immutable:** Objects that cannot be changed (e.g., int, str, tuple).

---

85. How do you convert data types in Python?

Answer:
Using built-in functions like int(), float(), str(), list(), tuple(), etc.

---

86. What is the __init__ method in Python?

Answer:
__init__ is the constructor method called automatically when an object is created; it initializes instance variables.

---

87. How do you handle multiple exceptions in Python?

Answer:
Using a tuple in the except block.

```
try:
    # code
except (TypeError, ValueError):
    # handle both exceptions
```

---

88. What is the difference between a generator and a list?

Answer:

- Generators yield items one at a time and are memory efficient.
- Lists store all items in memory.

---

89. How do you iterate over a dictionary?

Answer:
Using for key, value in dict.items():

---

90. What is a Python decorator?

Answer:
A decorator is a function that takes another function and extends its behavior without modifying its code, usually with the @decorator syntax.

91. What is the difference between del and remove() in Python lists?

Answer:

- del is a statement that deletes an element at a specific index or deletes variables completely.
- remove() is a list method that removes the first occurrence of a specified value.

Example:

```
lst = [1, 2, 3, 4]


del lst[1]  # Deletes element at index 1; lst becomes [1, 3, 4]


lst.remove(3)  # Removes first occurrence of value 3; lst becomes [1, 4]
```

---

92. What are Python's built-in functions to work with iterables?

Answer:
Important built-in functions include:

- all() – returns True if all elements are true.
- any() – returns True if any element is true.
- zip() – aggregates elements from multiple iterables.

- enumerate() – adds a counter to an iterable.
- filter() – filters elements based on a function.
- map() – applies a function to all elements.

---

93. How do you merge two dictionaries in Python?

Answer:
In Python 3.9+, you can use the | operator:

```
dict1 = {'a': 1}

dict2 = {'b': 2}

merged = dict1 | dict2  # {'a': 1, 'b': 2}
```

In earlier versions, use update() or unpacking:

```
merged = {**dict1, **dict2}
```

---

94. What is the difference between isinstance() and type()?

Answer:

- type() returns the exact type of the object.
- isinstance() checks if an object is an instance of a class or its subclass.

Example:

```
class A: pass

class B(A): pass


b = B()

print(type(b) == A)       # False

print(isinstance(b, A))   # True
```

---

95. What is the __call__ method in Python?

Answer:
The __call__ method allows an instance of a class to be called as a function.

Example:

```
class Callable:
    def __call__(self):
        print("Instance called")


c = Callable()
c()  # Output: Instance called
```

---

96. What are Python's comprehensions and generator expressions?

Answer:

- **Comprehensions:** concise syntax for creating lists, sets, or dicts.
- **Generator expressions:** similar syntax but return a generator that yields items lazily.

Example:

```
# List comprehension
lst = [x*x for x in range(5)]


# Generator expression
gen = (x*x for x in range(5))
```

---

97. Explain the with statement and context managers.

Answer:
The with statement simplifies exception handling and ensures resource cleanup. Context managers define __enter__ and __exit__ methods.

---

98. What is the difference between shallow copy and deep copy?

Answer:

- **Shallow copy:** copies the outer object but references nested objects.
- **Deep copy:** copies the outer object and recursively copies nested objects.

---

99. What is the difference between append() and extend() in lists?

Answer:

- append() adds a single element to the end of the list.
- extend() adds all elements from an iterable to the list.

Example:

```
lst = [1, 2]

lst.append([3, 4])  # [1, 2, [3, 4]]

lst.extend([5, 6])  # [1, 2, [3, 4], 5, 6]
```

---

100. How do you handle command-line arguments in Python?

Answer:
Using the sys.argv list or argparse module for more complex argument parsing.

Example:

```
import sys

print(sys.argv)  # List of command-line arguments
```

101. Explain Python's memory management and reference counting.

Answer:
Python manages memory automatically using **reference counting** and **garbage collection**. Each object keeps track of how many references point to it. When the reference count reaches zero, the memory is freed. Python also uses a cyclic garbage collector to detect and clean up reference cycles that reference counting alone can't handle.

---

102. What are Python's built-in decorators and how do you write custom decorators?

Answer:
Python has built-in decorators like @staticmethod, @classmethod, and @property. Decorators are functions that modify the behavior of other functions or methods.

Custom decorator example:

```
def decorator(func):
    def wrapper(*args, **kwargs):
        print("Before function call")
        result = func(*args, **kwargs)
        print("After function call")
        return result
    return wrapper

@decorator
def say_hello():
    print("Hello!")

say_hello()
```

---

103. How do you implement data serialization and deserialization beyond pickle?

Answer:
Besides pickle, Python supports JSON (json module) for human-readable serialization, marshal for internal use, and third-party libraries like yaml and msgpack.

---

104. What is the difference between @staticmethod and @classmethod with examples?

Answer:

- @staticmethod defines a method that does not receive an implicit first argument.
- @classmethod defines a method that receives the class (cls) as the first argument.

Example:

```
class MyClass:
    @staticmethod
    def static_method():
        print("Static method called")

    @classmethod
    def class_method(cls):
        print(f"Class method called from {cls}")

MyClass.static_method()
MyClass.class_method()
```

---

105. Explain Python's Global Interpreter Lock (GIL) and its impact on multithreading.

Answer:
The **GIL** is a mutex that allows only one thread to execute Python bytecode at a time. It limits true parallel execution of threads, especially for CPU-bound tasks. For I/O-bound tasks, multithreading can still be beneficial.

---

106. How can you speed up Python code? Discuss profiling tools and optimization techniques.

Answer:

- Use built-in functions and libraries
- Avoid unnecessary computations
- Profile code using cProfile, line_profiler, or timeit
- Use generators and lazy evaluation
- Use concurrency or multiprocessing for parallelism
- Use tools like Cython or PyPy for compilation

---

107. What are Python's generator functions and how do you use them for efficient data processing?

Answer:
Generators yield items one by one and are memory efficient for processing large data streams without loading everything into memory.

---

108. What is the difference between synchronous and asynchronous programming in Python?

Answer:

- **Synchronous:** Code runs sequentially; each operation waits for the previous one to finish.
- **Asynchronous:** Code can start a task and switch to another before the first finishes, useful for I/O-bound tasks.

---

109. How do you handle thread synchronization in Python?

Answer:
Use synchronization primitives like Lock, RLock, Semaphore, and Event from the threading module to avoid race conditions.

---

110. What are the best practices for writing clean and Pythonic code?

Answer:

- Follow PEP 8 style guide
- Use meaningful variable names
- Write modular, reusable functions
- Use list comprehensions and generators
- Avoid deep nesting
- Write docstrings and comments
- Handle exceptions properly

101. What is Python memory management?

Answer:
Python manages memory automatically with a **private heap** where all objects and data structures reside. The **Python Memory Manager** handles allocation, while **reference counting** tracks object usage. When reference counts reach zero, the memory is freed. Python also uses a **cyclic garbage collector** to clean up reference cycles.

---

102. What is packing and unpacking in Python function arguments?

Answer:

- **Packing:** Collecting multiple positional arguments into a tuple using *args, and keyword arguments into a dictionary using **kwargs.
- **Unpacking:** Passing elements of a list/tuple/dict as individual arguments using * and **.

Example:

```
def func(*args, **kwargs):
    print(args)
    print(kwargs)


func(1, 2, a=3, b=4)
# args: (1, 2)
# kwargs: {'a': 3, 'b': 4}


lst = [1, 2]
dct = {'a': 3}
func(*lst, **dct)
```

---

103. What are Python's mutable and immutable data types?

Answer:

- **Mutable:** Objects that can be changed after creation (e.g., list, dict, set).
- **Immutable:** Objects that cannot be changed after creation (e.g., int, float, str, tuple).

---

104. How does Python handle variable scope and namespaces?

Answer:
Python uses the **LEGB** rule for scope lookup:

- **Local:** Inside current function.
- **Enclosing:** In enclosing functions.
- **Global:** Module-level variables.
- **Built-in:** Python built-in names.

---

105. Explain Python's memory allocation for objects.

Answer:
Python allocates memory for objects in a private heap. Small integers and interned strings are cached to optimize performance. Reference counting and garbage collection manage object lifecycle and deallocation.

---

106. What is the difference between *args and **kwargs?

Answer:

- *args allows a function to accept any number of positional arguments (packed as a tuple).
- **kwargs allows accepting any number of keyword arguments (packed as a dictionary).

---

107. What is Python's garbage collection?

Answer:
Python's garbage collector frees memory by removing unused objects, especially those involved in cyclic references that cannot be freed by reference counting alone.

---

108. Explain shallow copy vs deep copy with examples.

Answer:

- **Shallow copy:** Copies the object but references nested objects.
- **Deep copy:** Copies the object and recursively copies all nested objects.

```
import copy


lst = [[1, 2], [3, 4]]

shallow = copy.copy(lst)

deep = copy.deepcopy(lst)
```

---

109. What is the difference between mutable and immutable function arguments?

Answer:
Mutable arguments (like lists) can be changed inside the function, affecting the original object. Immutable arguments (like ints or strings) cannot be changed inside the function.

---

110. What is Python's reference counting?

Answer:
Each object maintains a count of references pointing to it. When this count reaches zero, Python frees the object's memory.